INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-02: Estructuras de Selección
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero Software "B"
Alumnos participantes:	Bravo López Jordán Samuel Sánchez Lema Isaac Adrian ……..
Asignatura:	ALGORITMOS Y LÓGICA DE PROGRAMACIÓN
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

1.1 Objetivos

General:
Utilizar estructuras de selección en el análisis y diseño de algoritmos.

Específicos:

- Diseñar e implementar una variedad de menos de carácter interactivo utilizando las estructuras de control switch y bucles do-while, permitiendo que el usuario gestione múltiples opciones.
- Construir clases con atributos privados, métodos públicos con el propósito de generar entidades dentro del mundo digital mediante el uso de comportamientos lógicos y controlados.
- Emplear una variedad de algoritmos con el propósito de realizar cambios en variables numéricas, booleanas basados principalmente en condiciones determinadas, tales como la actualización de saldos en transferencias bancarias o la reducción progresiva de atributos físicos en procesos de lavado y secado, garantizando la integridad de la información.

1.2 Modalidad
Presencial.

1.3 Tiempo de duración
Presenciales: 8.
No presenciales: 0.

1.4 Instrucciones
- El trabajo se desarrollará de manera individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato PDF que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.
- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas.

1.5 Listado de equipos, materiales y recursos

- **Inteligencia artificial, TAC Inteligencia artificial, TAC Inteligencia artificial, TAC.**
- **- Computador o laptop.**
- **- Plataforma educativa de la UTA.**
- **- Internet para consultas. - Papel - Material de escritorio.**

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☒Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

1.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas.

1.7 Resultados obtenidos

Se ejecutó el diseño de un sistema basado en un menú interactivo. Se destaca la capacidad del algoritmo para realizar operaciones aritméticas sobre un arreglo de estructuras, manteniendo la integridad del saldo.

Modularidad: El uso de clases permitió que el código fuera más limpio y fácil de mantener en comparación con el enfoque puramente secuencial.
Interactividad: La implementación de estructuras de selección (switch e if) garantizó que el programa responda de forma coherente a las entradas del usuario, evitando errores de ejecución por opciones no válidas.
Memoria: Se comprobó que el uso de estructuras (struct) y clases (class) optimiza el manejo de datos relacionados bajo un mismo identificador.

1.8 Habilidades blandas empleadas en la práctica

☒ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☒ La empatía
☐ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

1.9 Conclusiones

Se determinó que el uso de las diferentes sentencias if-else además de switch es esencial para desarrollar programas participativos y fuertes. En el ejercicio del sistema bancario, estas estructuras permitieron validar condiciones críticas (como la existencia de saldo suficiente) antes de ejecutar una operación, garantizando que el flujo del algoritmo responda lógicamente a las entradas del usuario y evite estados inconsistentes en los datos.

La implementación de la clase Jean demostró que la POO facilita el mantenimiento y la escalabilidad del código. Al encapsular los datos (como la humedad y el color) y protegerlos bajo métodos específicos (lavar, secar), se logró una representación fiel de los procesos físicos de una fábrica o tienda, asegurando que los atributos solo se modifiquen mediante comportamientos predefinidos y no por accesos externos no autorizados.

A través del desarrollo del control de hora y los menús cíclicos (do-while), se concluye que la combinación de estructuras repetitivas con selectivas mejoras significativamente la experiencia del usuario. Esta metodología permite que el software sea flexible, dándole al usuario el control total sobre la manipulación de la información en tiempo real (modificar datos registrados o realizar múltiples transacciones) sin necesidad de reiniciar la ejecución del programa.

1.10 Recomendaciones

Se recomienda añadir mecanismos de control para las entradas del usuario, especialmente en el **Ejercicio de Control de Hora** y el **Sistema Bancario**. Utilizar estructuras if para verificar que las horas no superen 23 o que los montos de transferencia no sean negativos evita que el programa procese datos ilógicos que podrían causar fallos en la ejecución o resultados erróneos.

Al trabajar con ejercicios como el de la clase **Jean**, es fundamental mantener los atributos como private. Se sugiere utilizar siempre métodos *getters* y *setters* para acceder o modificar la información. Esto no solo protege la integridad de los datos (evitando, por ejemplo, que la humedad sea menor a 0%), sino que también facilita la transición de la lógica desde otros lenguajes como Java hacia C++.

1.11 Referencias bibliográficas

I. Sánchez y S. Bravo, "ALGORITMOS-Y-LOGICA-DE-PROGRAMACION-SW-B-APE-2," GitHub Repository, 2026. [En línea]. Disponible en: https://github.com/isma1402uta-boop/ALGORITMOS-Y-LOGICA-DE-PROGRAMACION-SW-B-APE-2. [Accedido: 30-mar-2026].

Ejercicios a realizar:

1) Ejercicio: Control de hora

Desarrollar un programa en **C++** que permita ingresar una hora, minutos y segundos. Luego, el sistema debe mostrar la hora registrada y preguntar al usuario si desea cambiarla. Si responde que sí, se debe permitir modificar la hora y volver a mostrarla. Si responde que no, el programa finaliza. Este enunciado se basa en el flujo del programa en c++ donde se ingresan horas, minutos y segundos, se muestran y luego se consulta si se desea cambiar la hora.

- **Análisis del problema**

El objetivo es crear un programa que gestione la entrada de tiempo (horas, minutos y segundos). El sistema debe ser cíclico: tras capturar y mostrar la hora, debe preguntar al usuario si desea corregir o cambiar los datos. La validación es implícita en el flujo, permitiendo la sobrescritura de las variables hasta que el usuario decida finalizar.

- **Variables de entrada, proceso y salida**

Tabla I. Variables del ejercicio 1

Categoría	Variable	Tipo de dato	Rango
Entrada	**horas**	**Int**	**0-23**
Entrada	**minutos**	**Int**	**0-59**
Entrada	**segundos**	**Int**	**0-59**
Entrada	**respuesta**	**Char**	**S, s, n, N**
Proceso	**validacion**	**Boolean**	**H<0, h>23**
Salida	**hora_formateada**	**string**	**HH:MM:SS**

- **Algoritmos (pSeInt)**

Definir variables para hora, minutos, segundos y la opción de repetir.

Iniciar un ciclo de repetición.

Solicitar al usuario el ingreso de la hora.

Solicitar al usuario el ingreso de los minutos.

Solicitar al usuario el ingreso de los segundos.

Mostrar la hora en pantalla.

Preguntar: "¿Desea cambiar la hora? (S/N)".

Si la respuesta es 'S' o 's', volver al paso 3.

Si la respuesta es 'N' o 'n', terminar el programa.

- **Pseudocódigo (pSeInt)**

```
Algoritmo ControlDeHora
  Definir horas, minutos, segundos Como Entero
  Definir respuesta Como Caracter

  Repetir
    Escribir "Ingrese las horas (0-23):"
    Leer horas
    Escribir "Ingrese los minutos (0-59):"
    Leer minutos
    Escribir "Ingrese los segundos (0-59):"
    Leer segundos

    Escribir "La hora registrada es: ", horas, ":", minutos, ":", segundos

    Escribir "¿Desea cambiar la hora? (S/N)"
```

Leer respuesta

Hasta Que respuesta == 'N' o respuesta == 'n'

Escribir "Programa finalizado."

FinAlgoritmo

- **Diagrama de flujo (pSeInt)**

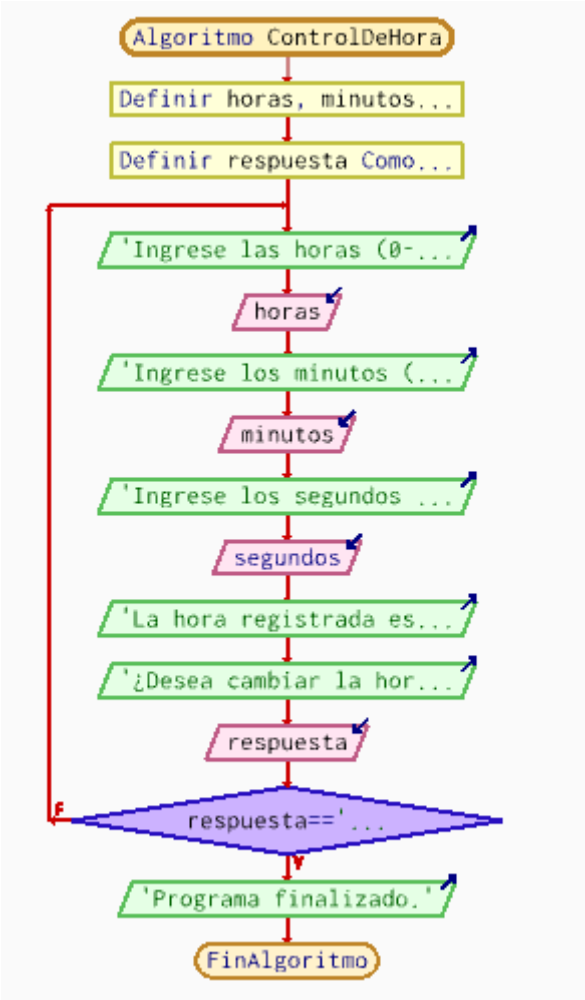


Ilustración 1. Diagrama de flujo del ejercicio 1

- **Código en C++ (codeblocs)**

```
#include <iostream>


using namespace std;
```

```
int main() {
    int horas, minutos, segundos;
    char respuesta;

    do {
        cout << "\n--- Registro de Hora (Validado) ---" << endl;

        // Bucle para validar horas
        do {
            cout << "Ingrese horas (0-23): ";
            cin >> horas;
            if (horas < 0 || horas > 23) {
                cout << " ERROR: El rango permitido es 0-23. Intente de nuevo." << endl;
            }
        } while (horas < 0 || horas > 23);

        // Bucle para validar minutos
        do {
            cout << "Ingrese minutos (0-59): ";
            cin >> minutos;
            if (minutos < 0 || minutos > 59) {
                cout << " ERROR: El rango permitido es 0-59." << endl;
            }
        } while (minutos < 0 || minutos > 59);
```

```
        // Bucle para validar segundos
        do {
            cout << "Ingrese segundos (0-59): ";
            cin >> segundos;
            if (segundos < 0 || segundos > 59) {
                cout << " ERROR: El rango permitido es 0-59." << endl;
            }
        } while (segundos < 0 || segundos > 59);

        cout << "\nHora registrada correctamente: " << horas << ":" << minutos << ":" <<
segundos << endl;

        cout << "\n¿Desea cambiar la hora registrada? (s/n): ";
        cin >> respuesta;

    } while (respuesta == 's' || respuesta == 'S');

    cout << "Sistema cerrado correctamente." << endl;

    return 0;
}
```

- **3 casos de prueba explicados**

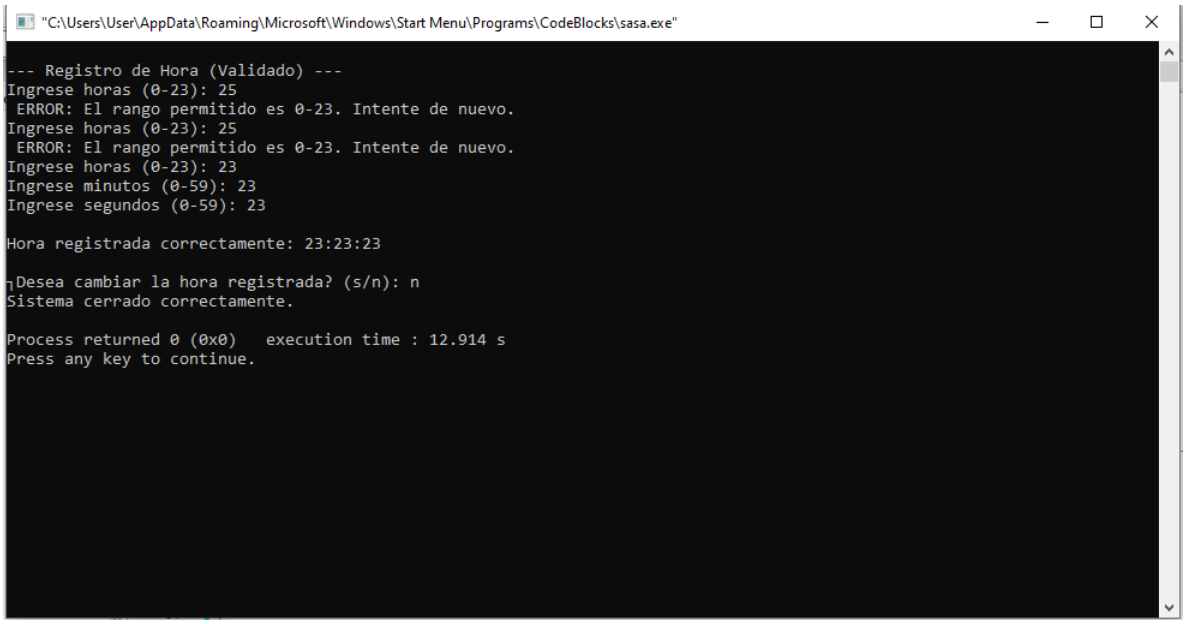


Ilustración 2. Primera prueba de caso del ejercicio 1

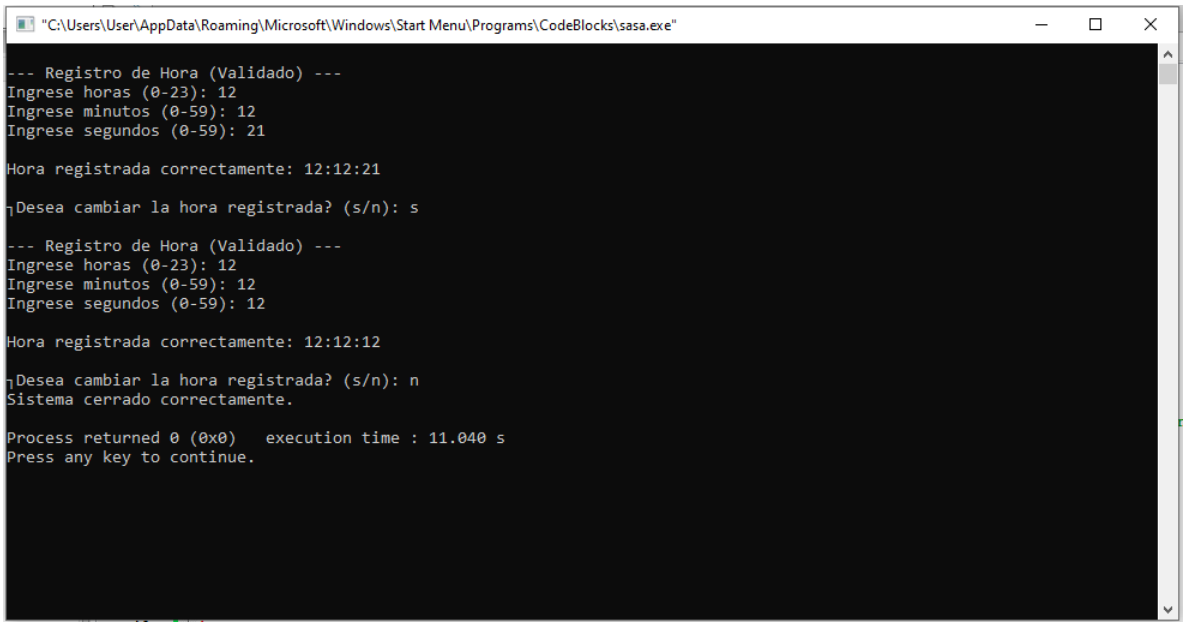


Ilustración 3. Segunda prueba de caso del ejercicio 1

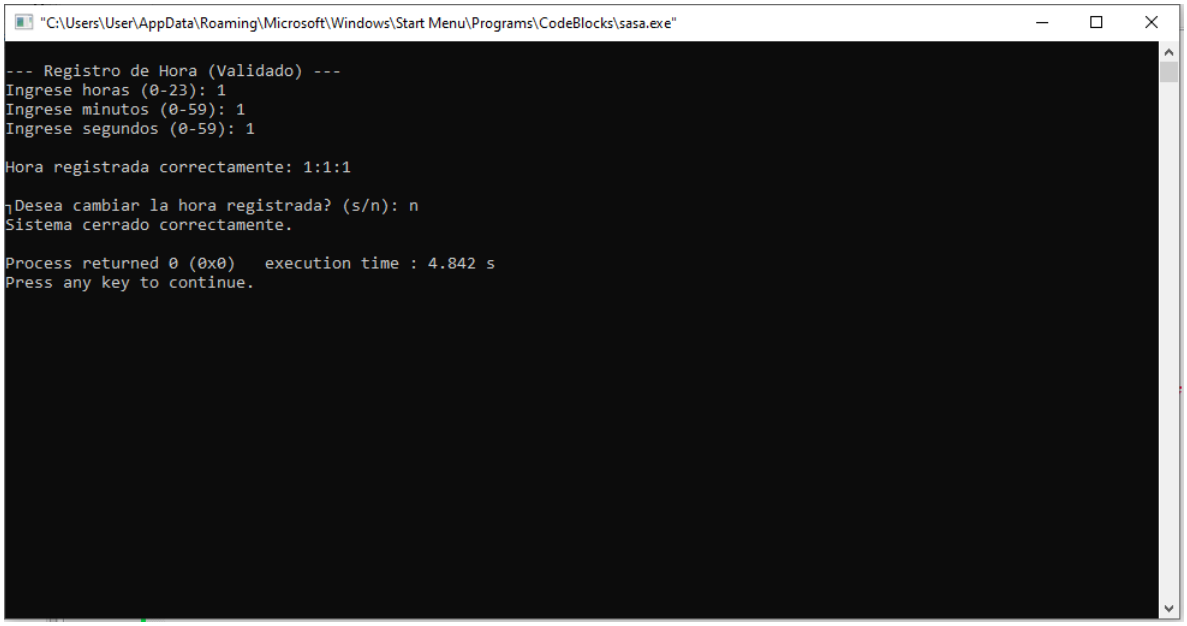


Ilustración 4. Tercera prueba de caso del ejercicio 1

2) Ejercicio: Sistema de cuentas bancarias

Desarrollar un programa en **C++** que permita registrar un cliente mediante su DNI y crear hasta tres cuentas bancarias. El usuario podrá seleccionar una cuenta y realizar acciones desde un menú: ver atributos de la cuenta, enviar dinero, recibir dinero, salir del sistema o hacer una transferencia entre cuentas.

- **Análisis del problema**

Se requiere un sistema que vincule a un cliente (identificado por DNI) con hasta tres cuentas bancarias. Cada cuenta debe ser una instancia con saldo y funciones propias. El núcleo del programa es un menú interactivo que permita la navegación entre cuentas y la ejecución de operaciones financieras básicas (depósitos, retiros y transferencias entre las cuentas del mismo usuario).

- **Variables de entrada, proceso y salida**

Tabla II. Variables del ejercicio 2

Categoría	Variable	Tipo de dato	Rango
Clase	**Saldo**	**double**	**>=0**
Clase	**IdCuenta**	**Int**	**1, 2 o 3**
Entrada	**Dni**	**String**	**N/A**
Entrada	**Opción**	**Int**	**1 a 2**
Entrada	**sel/dest**	**Int**	**1 a 3**
Entrada	**sub0p**	**String**	**1 a 5**
Entrada	**Monto**	**Doublé**	**Valor > 0**
Proceso	**Idx/idxDest**	**Int**	**0 a 2**
Proceso	**retirar**	**Boolean**	**monto<<saldo**
Salida	**Mensajes**	**String**	**Mensaje**

- **Algoritmos (pSeInt)**

Solicitar DNI del cliente.

Inicializar 3 cuentas con saldos iniciales (pueden empezar en 0).

Mostrar menú principal: 1. Seleccionar Cuenta, 2. Salir.

Si selecciona cuenta, mostrar sub-menú de operaciones:

Ver atributos.

Enviar dinero (Retiro).

Recibir dinero (Depósito).

Transferencia (Mover entre las 3 cuentas).

Repetir hasta que el usuario decida salir.

- **Pseudocódigo (pSeInt)**

```
Algoritmo SistemaBancario
   // 1. Definicion de variables
   Definicion dni Como Cadena
   Dimension saldos[3] // Arreglo para 3 cuentas
   Definir saldos Como Real
   Definir opcion, cuentaSel, cuentaDest, subOpcion Como Entero
   Definir monto Como Real

   // 2. Inicializacion
   Escribir "--- Registro de Cliente ---"
```

```
Escribir "Ingrese DNI del cliente:"
Leer dni

// Inicializar las 3 cuentas en 0
saldos[1] <- 0
saldos[2] <- 0
saldos[3] <- 0

Repetir
   Escribir ""
   Escribir "--- MENU PRINCIPAL (DNI: ", dni, ") ---"
   Escribir "1. Seleccionar/Gestionar Cuenta"
   Escribir "2. Salir del Sistema"
   Escribir "Elija una opcion:"
   Leer opcion

   Si opcion = 1 Entonces
      Escribir "Seleccione numero de cuenta (1, 2 o 3):"
      Leer cuentaSel

      // Validar que la cuenta exista
      Si cuentaSel >= 1 Y cuentaSel <= 3 Entonces
         Repetir
            Escribir ""
            Escribir ">> GESTIONANDO CUENTA ", cuentaSel
```

```
Escribir "1. Ver Atributos (Saldo)"
Escribir "2. Enviar Dinero (Retirar)"
Escribir "3. Recibir Dinero (Depositar)"
Escribir "4. Transferencia entre cuentas"
Escribir "5. Volver al Menu Principal"
Leer subOpcion

Segun subOpcion Hacer
  1:
    Escribir "ID Cuenta: ", cuentaSel, " | Saldo Actual: $", saldos[cuentaSel]
  2:
    Escribir "Ingrese monto a retirar:"
    Leer monto
    Si monto <= saldos[cuentaSel] Entonces
      saldos[cuentaSel] <- saldos[cuentaSel] - monto
      Escribir "Retiro exitoso."
    Sino
      Escribir "ERROR: Fondos insuficientes."
    FinSi
  3:
    Escribir "Ingrese monto a depositar:"
    Leer monto
    saldos[cuentaSel] <- saldos[cuentaSel] + monto
    Escribir "Deposito exitoso."
```

```
4:
    Escribir "Seleccione cuenta destino (1, 2 o 3):"
    Leer cuentaDest
    Si cuentaDest >= 1 Y cuentaDest <= 3 Y cuentaDest <> cuentaSel
Entonces
        Escribir "Monto a transferir:"
        Leer monto
        Si monto <= saldos[cuentaSel] Entonces
            saldos[cuentaSel] <- saldos[cuentaSel] - monto
            saldos[cuentaDest] <- saldos[cuentaDest] + monto
            Escribir "Transferencia enviada con exito."
        Sino
            Escribir "ERROR: Fondos insuficientes."
        FinSi
    Sino
        Escribir "ERROR: Cuenta destino invalida o igual a la actual."
    FinSi
  FinSegun
 Hasta Que subOpcion = 5
Sino
 Escribir "ERROR: La cuenta ingresada no existe."
FinSi
FinSi

Hasta Que opcion = 2
```

Escribir "Gracias por usar el sistema bancario. Saliendo..."

FinAlgoritmo

- **Diagrama de flujo (pSeInt)**

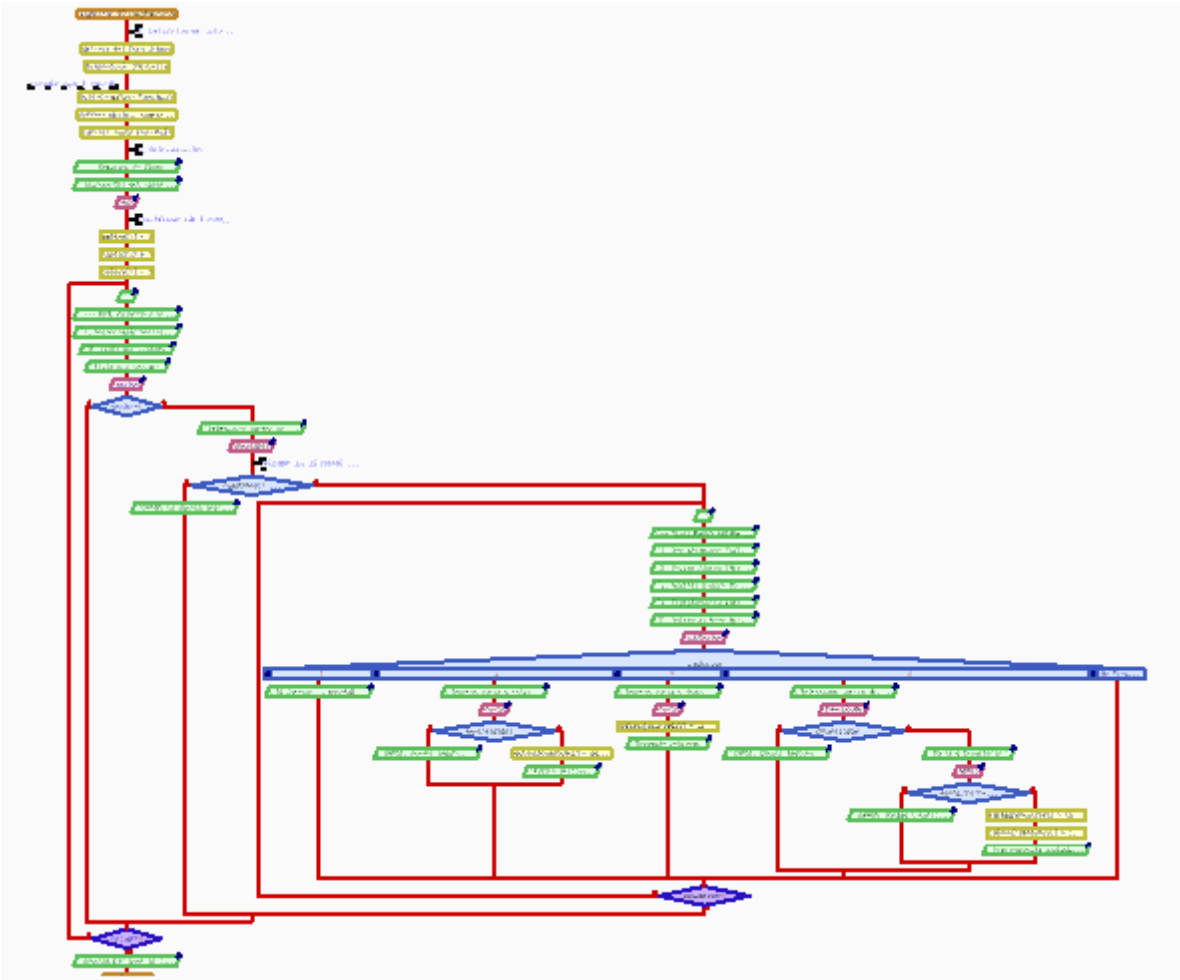


Ilustración 5. Diagrama de flujo del ejercicio 2

- **Código en C++ (codeblocs)**

```
#include <iostream>

#include <string>

#include <vector>


using namespace std;
```

```
// Definición de la Clase
class CuentaBancaria {
private:
    double saldo;
    int idCuenta;

public:
    // Constructor
    CuentaBancaria() {
        saldo = 0.0;
    }

    // Métodos (Comportamientos)
    void establecerID(int id) { idCuenta = id; }

    double obtenerSaldo() { return saldo; }

    void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
            cout << "[!] Deposito exitoso. Nuevo saldo: $" << saldo << endl;
        }
    }
```

```
    bool retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            cout << "[!] Retiro exitoso. Saldo restante: $" << saldo << endl;
            return true;
        } else {
            cout << "[X] Error: Fondos insuficientes o monto invalido." << endl;
            return false;
        }
    }

    void mostrarAtributos() {
        cout << "-------------------------" << endl;
        cout << "ID Cuenta: " << idCuenta << endl;
        cout << "Saldo Disponible: $" << saldo << endl;
        cout << "Estado: Activa" << endl;
        cout << "-------------------------" << endl;
    }
};

int main() {
    string dni;
    CuentaBancaria misCuentas[3]; // Arreglo de objetos (clases)
    int opcion, sel, dest, subOp;
    double monto;
```

```
// Inicialización de IDs para las cuentas
for(int i = 0; i < 3; i++) {
    misCuentas[i].establecerID(i + 1);
}

cout << "=== REGISTRO DE CLIENTE ===" << endl;
cout << "Ingrese DNI: ";
cin >> dni;

do {
    cout << "\n--- BANCO (Cliente: " << dni << ") ---" << endl;
    cout << "1. Gestionar Cuentas\n2. Salir\nSeleccion: ";
    cin >> opcion;

    if (opcion == 1) {
        cout << "\nSeleccione cuenta (1, 2 o 3): ";
        cin >> sel;
        int idx = sel - 1; // Ajuste para el arreglo (0-2)

        if (idx >= 0 && idx < 3) {
            do {
                cout << "\n>> MENU CUENTA " << sel << endl;
                cout << "1. Ver atributos\n2. Enviar (Retirar)\n3. Recibir (Depositar)\n4. Transferencia\n5. Volver\nOp: ";
```

```
cin >> subOp;


switch(subOp) {
    case 1: misCuentas[idx].mostrarAtributos(); break;
    case 2:
        cout << "Monto a retirar: "; cin >> monto;
        misCuentas[idx].retirar(monto);
        break;
    case 3:
        cout << "Monto a depositar: "; cin >> monto;
        misCuentas[idx].depositar(monto);
        break;
    case 4:
        cout << "Seleccione cuenta destino (1-3): "; cin >> dest;
        int idxDest = dest - 1;
        if (idxDest >= 0 && idxDest < 3 && idxDest != idx) {
            cout << "Monto a transferir: "; cin >> monto;
            if (misCuentas[idx].retirar(monto)) {
                misCuentas[idxDest].depositar(monto);
                cout << "[!] Transferencia realizada con exito." << endl;
            }
        } else {
            cout << "[X] Cuenta destino no valida." << endl;
        }
        break;
```

```
                }
            } while (subOp != 5);
        } else {
            cout << "[X] Numero de cuenta invalido." << endl;
        }
    }
} while (opcion != 2);


cout << "Gracias por preferir nuestro banco." << endl;
return 0;
}
```

- **3 casos de prueba explicados**

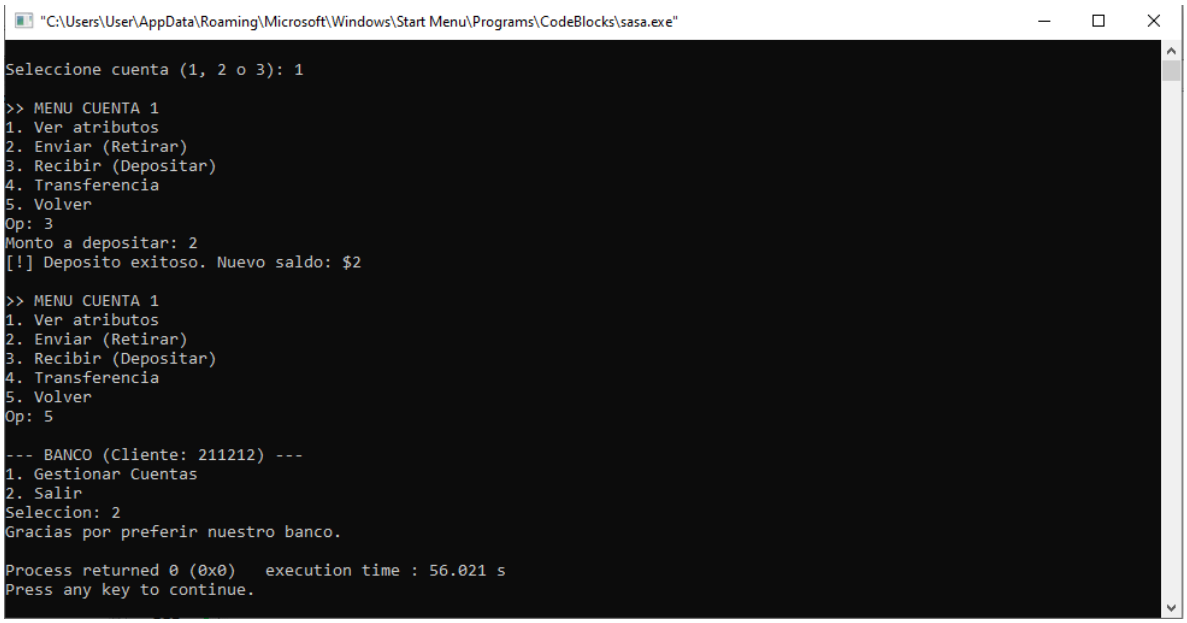


Ilustración 6. Primera prueba de caso del ejercicio 2

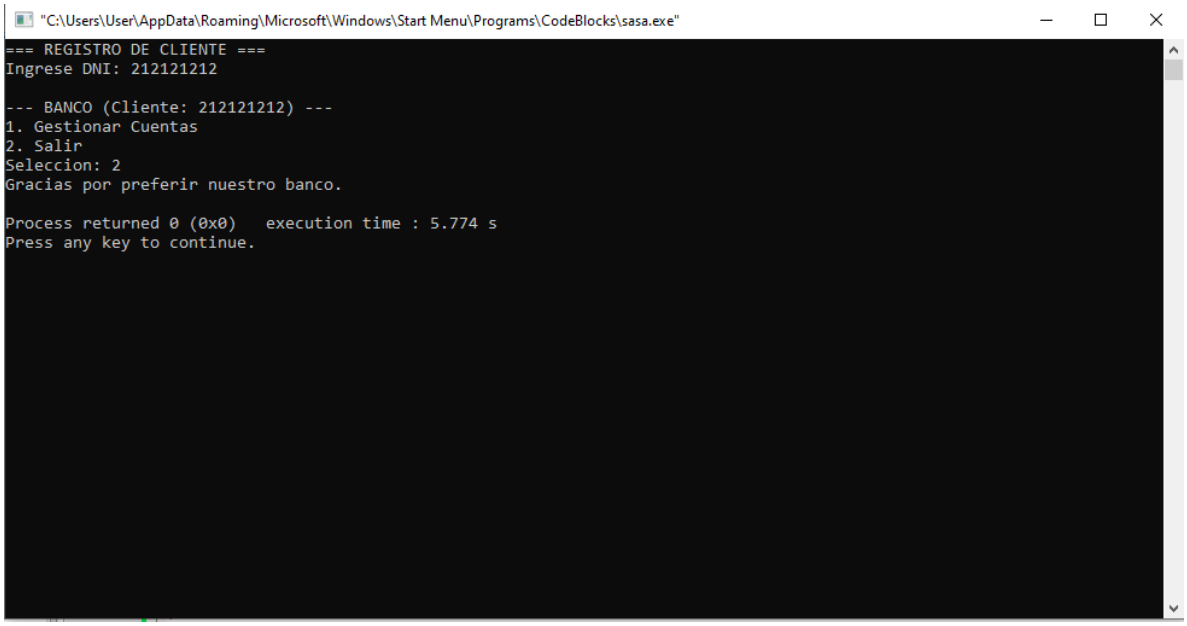


Ilustración 7. Segunda prueba de caso del ejercicio 2

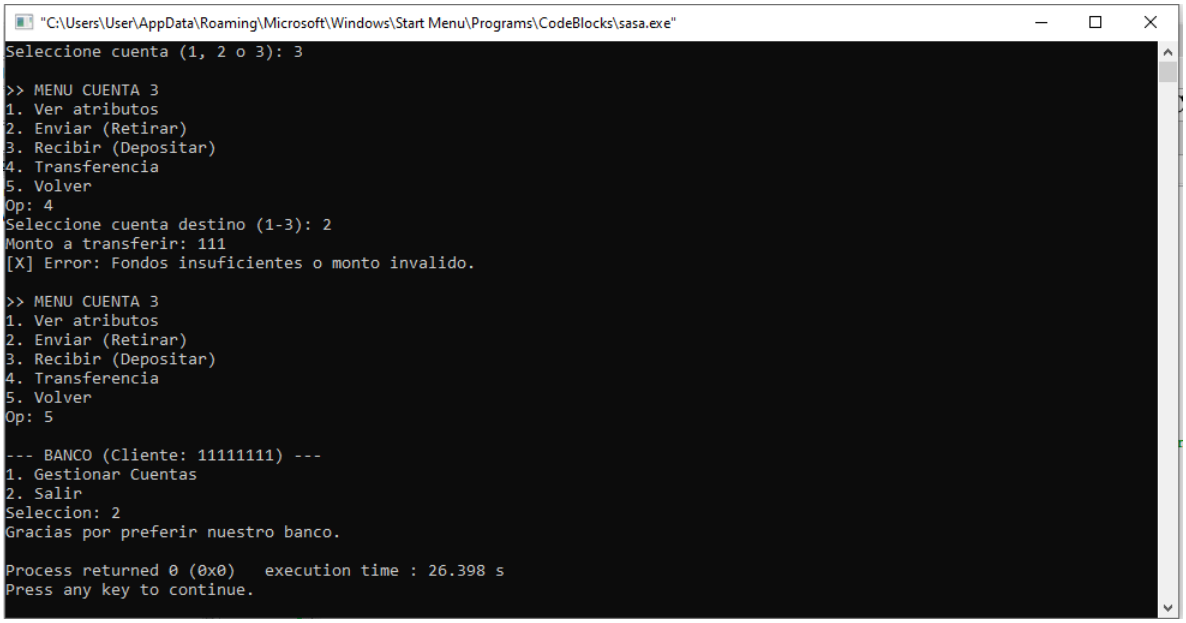


Ilustración 8. Tercera prueba de caso del ejercicio 2

3) Ejercicio: Gestión de un jean en fábrica o tienda

Desarrollar un programa en **C++** que permita registrar los datos de un jean: código, color, talla, si fue teñido, cantidad de teñidos, precio, cantidad de botones, humedad y estado de la tela. El sistema debe tener opciones para mostrar los datos del jean, lavarlo y secarlo. Al lavar, disminuye la cantidad de teñidos; al secar, disminuye la humedad. Este enunciado se basa en la clase Jean.java, que contiene esos atributos y los métodos Lavar, Secar y Mostrar Datos.

- **Análisis del problema**

El sistema busca administrar el ciclo de mantenimiento de una prenda (Jean) en un entorno de fábrica o tienda. El núcleo del problema consiste en transformar atributos estáticos en estados dinámicos que reaccionen a las acciones del usuario (Lavar y Secar).

Para resolver el problema mediante Programación Orientada a Objetos, se identifican:

- Atributos de Identificación: Código, color, talla, precio y cantidad de botones. Son valores que definen la identidad de la prenda.
- Atributos de Estado: * cantidadTeñidos: Representa la durabilidad del color.

humedad: Representa el estado físico de la tela (mojada/seca).

Las reglas de negocio dictan el comportamiento de las funciones principales:

- Método Lavar: Actúa como un proceso de desgaste y saturación. Debe restar 1 a la cantidad de teñidos (siempre que sea mayor a 0) y elevar la humedad al 100% de forma automática.

- Método Secar: Actúa como un proceso de restauración. Debe restar un valor porcentual constante (ej. 25%) a la humedad. Se requiere una validación de límite para asegurar que la humedad nunca sea menor a 0%.
- Método MostrarDatos: Actúa como una función de reporte, imprimiendo en consola el estado actual de todas las variables del objeto.

Se requiere un menú iterativo basado en una estructura Hacer-Mientras (Do-While). Esto permite que una misma instancia de la clase sea afectada por múltiples procesos consecutivamente, permitiendo observar la evolución de los datos hasta que el usuario decida finalizar.

- **Variables de entrada, proceso y salida**

Tabla III. Variables del ejercicio 3

Categoría	Variable	Tipo de dato	Rango
Clase	**codigo**	**string**	**N/A**
Clase	**Color**	**string**	**N/A**
Clase	**Talla**	**String**	**S,M,L,XL**
Clase	**CantidadTenidos**	**Int**	**>=0**
Clase	**Humedad**	**Double**	**0 a 100**
Clase	**precio**	**Double**	**>0**
Clase	**Botones**	**Int**	**> 0**
Entrada	**Opción**	**Int**	**1 a 4**
Proceso	**Lavar**	**Void**	**Tenidos -1**
Proceso	**Secar**	**Void**	**Humedad -25**
Salida	**MostrarDatos**	**Void**	**N/A**

- **Algoritmos (pSeInt)**

Definir Atributos: Crear variables para código, color, talla, teñidos, precio, botones y humedad.

Inicializar Datos: Asignar valores de fábrica a la prenda (ej. Humedad 10%, Teñidos 5).

Ciclo de Menú: Repetir las siguientes opciones hasta que el usuario elija "Salir":

Opción 1 (Mostrar): Listar todos los atributos actuales en pantalla.

Opción 2 (Lavar): Restar **1** a cantidad de teñidos (si es > 0) y fijar humedad al **100%**.

Opción 3 (Secar): Restar un **20%** a la humedad actual (evitando valores negativos).

Opción 4 (Salir): Finalizar la ejecución del programa.

- **Pseudocódigo (pSeInt)**

```
Algoritmo GestionDeJean
    // 1. Definicion de variables de entrada y proceso
    Definir codigo, color, talla, estadoTela Como Cadena
    Definir cantidadTenidos, botones, opcion Como Entero
    Definir precio, humedad Como Real

    // 2. Inicializacion de datos (Valores iniciales del objeto)
    codigo <- "JN-T502"
    color <- "Azul Indigo"
    talla <- "32"
    cantidadTenidos <- 5
    precio <- 45.99
    botones <- 4
```

```
humedad <- 15.0
estadoTela <- "Nueva"

// 3. Ciclo de interaccion (Menu)
Repetir
    Escribir ""
    Escribir "======= MENU GESTION TEXTIL ======="
    Escribir "1. Mostrar Datos del Jean"
    Escribir "2. Lavar (Desgasta tinte y moja)"
    Escribir "3. Secar (Disminuye humedad)"
    Escribir "4. Salir"
    Escribir "Seleccione una opcion (1-4):"
    Leer opcion

    Segun opcion Hacer
        1:
            Escribir "----------------------------------"
            Escribir "CODIGO: ", codigo
            Escribir "COLOR: ", color, " | TALLA: ", talla
            Escribir "TEÑIDOS RESTANTES: ", cantidadTenidos
            Escribir "HUMEDAD: ", humedad, "%"
            Escribir "ESTADO DE TELA: ", estadoTela
            Escribir "PRECIO: $", precio
            Escribir "----------------------------------"
        2:
```

```
            // Logica de Lavado
            Si cantidadTenidos > 0 Entonces
                cantidadTenidos <- cantidadTenidos - 1
                Escribir "[!] Lavado: El color se ha desgastado."
            Sino
                Escribir "[!] Advertencia: No queda mas tinte."
            FinSi
            humedad <- 100 // Al lavar siempre se moja al maximo
            Escribir "[!] El Jean ahora esta totalmente mojado."
        3:
            // Logica de Secado
            Si humedad > 0 Entonces
                humedad <- humedad - 20
                Si humedad < 0 Entonces
                    humedad <- 0
                FinSi
                Escribir "[!] Secando... Humedad actual: ",
humedad, "%"
            Sino
                Escribir "[!] El Jean ya esta seco (0% humedad)."
            FinSi
        4:
            Escribir "Cerrando programa de gestion..."
        De Otro Modo:
            Escribir "Error: Opcion no valida."
```

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

- **Diagrama de flujo (pSeInt)**

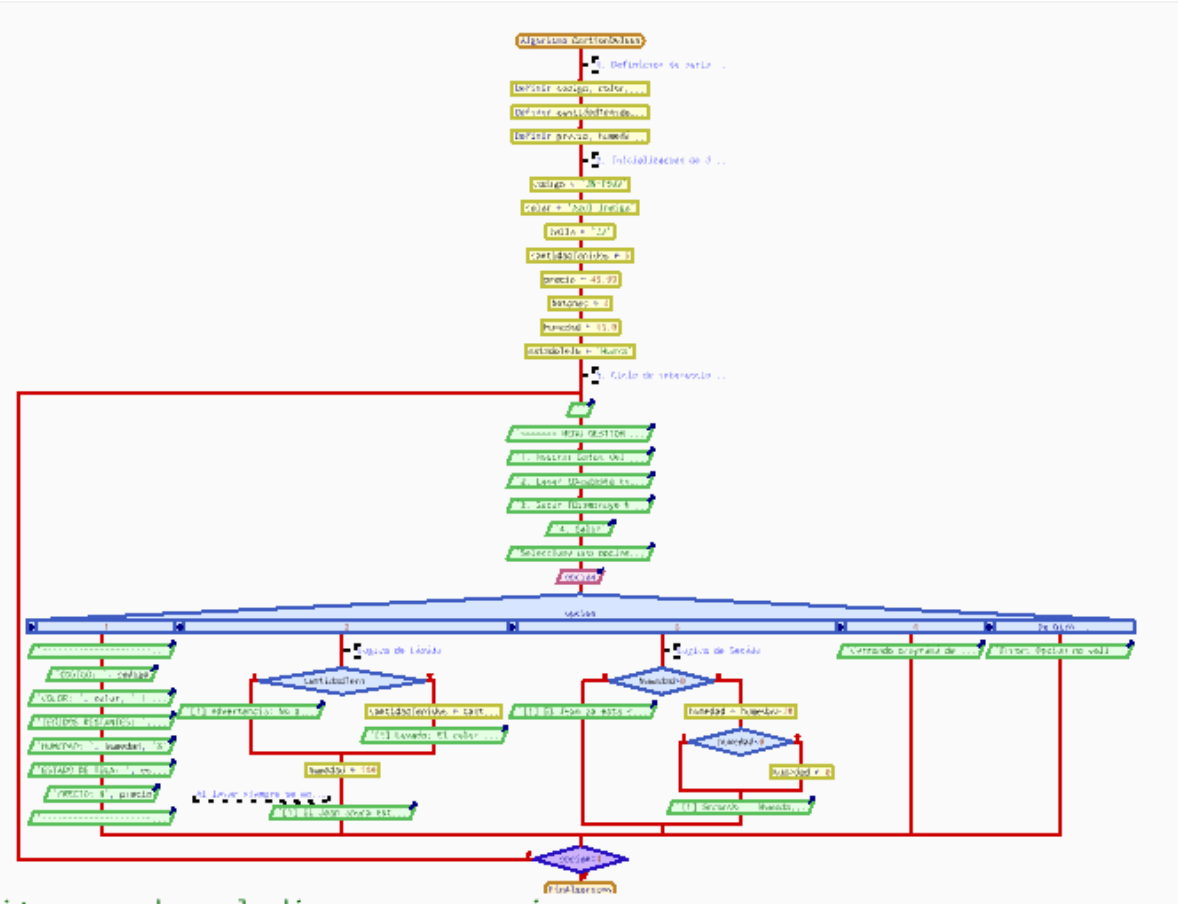


Ilustración 9. Diagrama de flujo del ejercicio 3

- **Código en C++ (codeblocs)**

```
#include <iostream>

#include <string>


using namespace std;


// Definición de la Clase
```

```
class Jean {
private:
    // Atributos privados
    string codigo, color, talla, estadoTela;
    int cantidadTenidos, botones;
    double precio, humedad;
    bool fueTenido;

public:
    // Constructor para inicializar el Jean
    Jean(string _cod, string _col, string _talla, int _tenidos, double _prec, int _bot) {
        codigo = _cod;
        color = _col;
        talla = _talla;
        cantidadTenidos = _tenidos;
        precio = _prec;
        botones = _bot;
        humedad = 10.0; // Humedad inicial por defecto
        estadoTela = "Nueva";
        fueTenido = (cantidadTenidos > 0);
    }

    // Método para Lavar
    void Lavar() {
        if (cantidadTenidos > 0) {
```

```
            cantidadTenidos--; // Disminuye el teñido
        }
        humedad = 100.0; // Se moja completamente
        cout << "\n[!] Lavado finalizado: Tinte desgastado y tela mojada (100%)." << endl;
    }

    // Método para Secar
    void Secar() {
        if (humedad > 0) {
            humedad -= 25.0; // Disminuye humedad
            if (humedad < 0) humedad = 0;
            cout << "\n[!] Secado finalizado: Humedad actual al " << humedad << "%." <<
endl;
        } else {
            cout << "\n[!] El jean ya esta totalmente seco (0%)." << endl;
        }
    }

    // Método para Mostrar Datos
    void MostrarDatos() {
        cout << "\n========================================" << endl;
        cout << "        DATOS ACTUALES DEL JEAN        " << endl;
        cout << "========================================" << endl;
        cout << "Codigo: " << codigo << " | Color: " << color << endl;
        cout << "Talla: " << talla << " | Botones: " << botones << endl;
```

```
        cout << "Cantidad de Tenidos: " << cantidadTenidos << endl;
        cout << "Humedad de la tela: " << humedad << "%" << endl;
        cout << "Precio: $" << precio << endl;
        cout << "Estado: " << estadoTela << endl;
        cout << "========================================" << endl;
    }
};

int main() {
    // Instancia del objeto miJean
    Jean miJean("J-2026", "Azul Indigo", "M", 5, 45.99, 4);
    int opcion;

    do {
        cout << "\n--- MENU GESTION DE JEAN ---" << endl;
        cout << "1. Mostrar Datos del Jean" << endl;
        cout << "2. Lavar" << endl;
        cout << "3. Secar" << endl;
        cout << "4. Salir" << endl;
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch(opcion) {
            case 1:
                miJean.MostrarDatos();
```

```
            break;
        case 2:
            miJean.Lavar();
            break;
        case 3:
            miJean.Secar();
            break;
        case 4:
            cout << "Cerrando sistema de fabrica..." << endl;
            break;
        default:
            cout << "Opcion no valida." << endl;
    }
  } while (opcion != 4);

  return 0;
}
```

- **3 casos de prueba explicados**

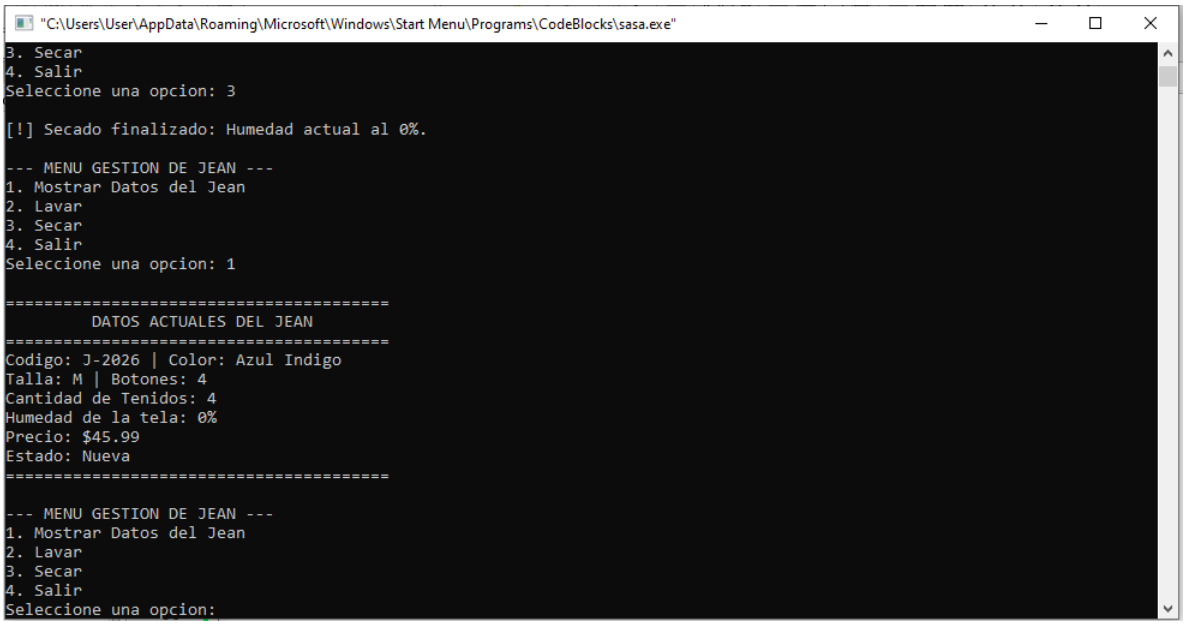


Ilustración 10.Primera prueba de caso del ejercicio 3

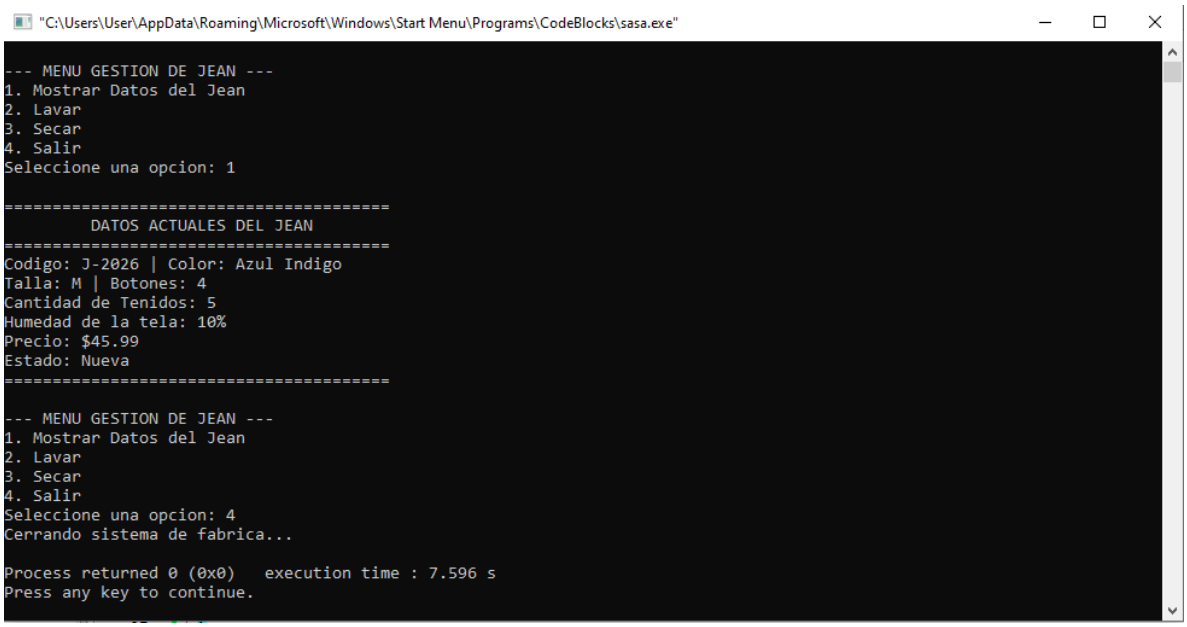
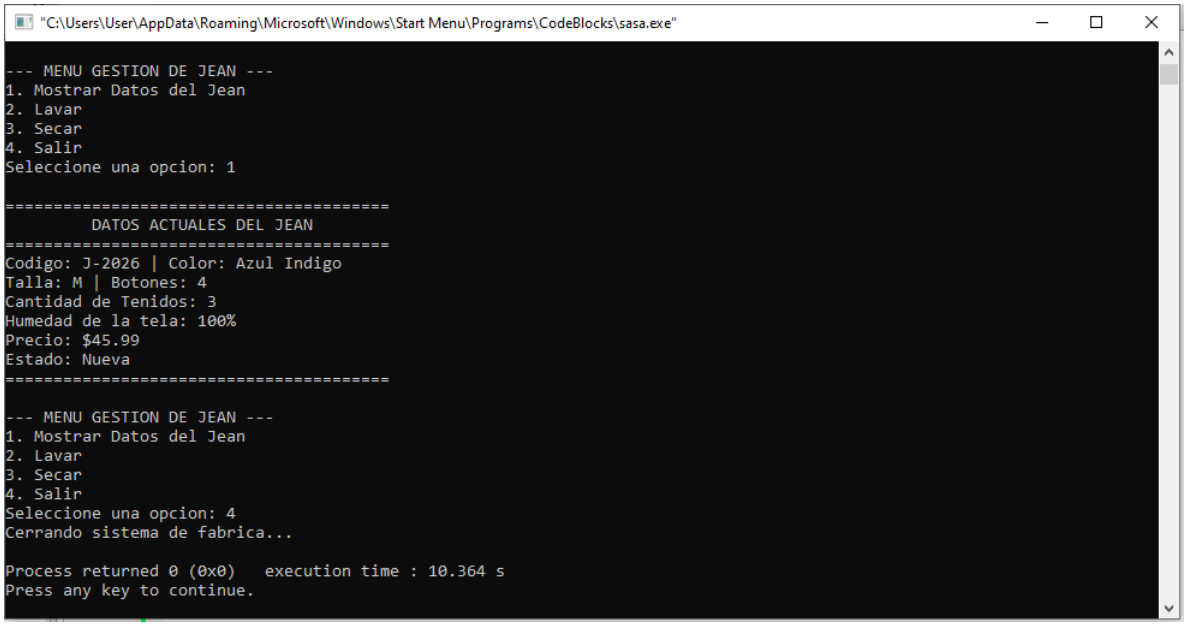


Ilustración 11. Segunda prueba de caso del ejercicio 3



```
"C:\Users\User\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\CodeBlocks\sasa.exe"

--- MENU GESTION DE JEAN ---
1. Mostrar Datos del Jean
2. Lavar
3. Secar
4. Salir
Seleccione una opcion: 1

========================================
         DATOS ACTUALES DEL JEAN
========================================
Codigo: J-2026 | Color: Azul Indigo
Talla: M | Botones: 4
Cantidad de Tenidos: 3
Humedad de la tela: 100%
Precio: $45.99
Estado: Nueva
========================================

--- MENU GESTION DE JEAN ---
1. Mostrar Datos del Jean
2. Lavar
3. Secar
4. Salir
Seleccione una opcion: 4
Cerrando sistema de fabrica...

Process returned 0 (0x0)   execution time : 10.364 s
Press any key to continue.
```

Ilustración 12. Tercera prueba de caso del ejercicio 3